

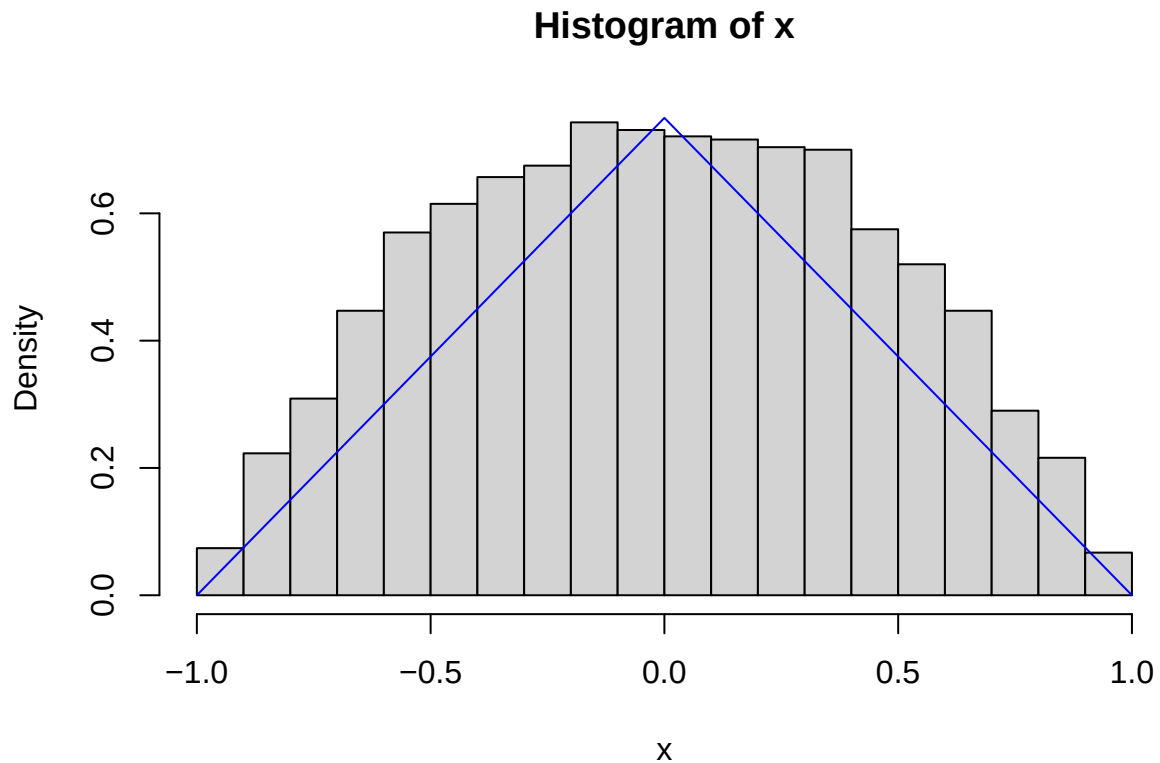
433-HW8

Alex Caramanico

November 3, 2022

Problem 1

```
# 1a
n = 10000
u1 = runif(n, min=-1, max=1) # generate random numbers
u2 = runif(n, min=-1, max=1)
u3 = runif(n, min=-1, max=1)
x = rep(0,n)
for (i in 1:n) { # algorithm to create values of x
  if (abs(u3[i]) >= abs(u2[i]) & abs(u3[i]) >= abs(u1)[i]) {
    x[i] = u2[i] # first condition
  }
  else {
    x[i] = u3[i] # otherwise condition
  }
}
z = seq(-1,1,01)
hist(x,freq=0) # plot histogram of x
lines(z, 3/4*(1-z^2),col='blue') # blue line is the actual density
```



```
# 1b see attached scan
# 1c
cat('The mean of the generated numbers is', mean(x), '\n') # should be ~0.0
```

```
## The mean of the generated numbers is -0.005164305
```

```
cat('The variance of the generated numbers is', var(x)) # should be ~0.2
```

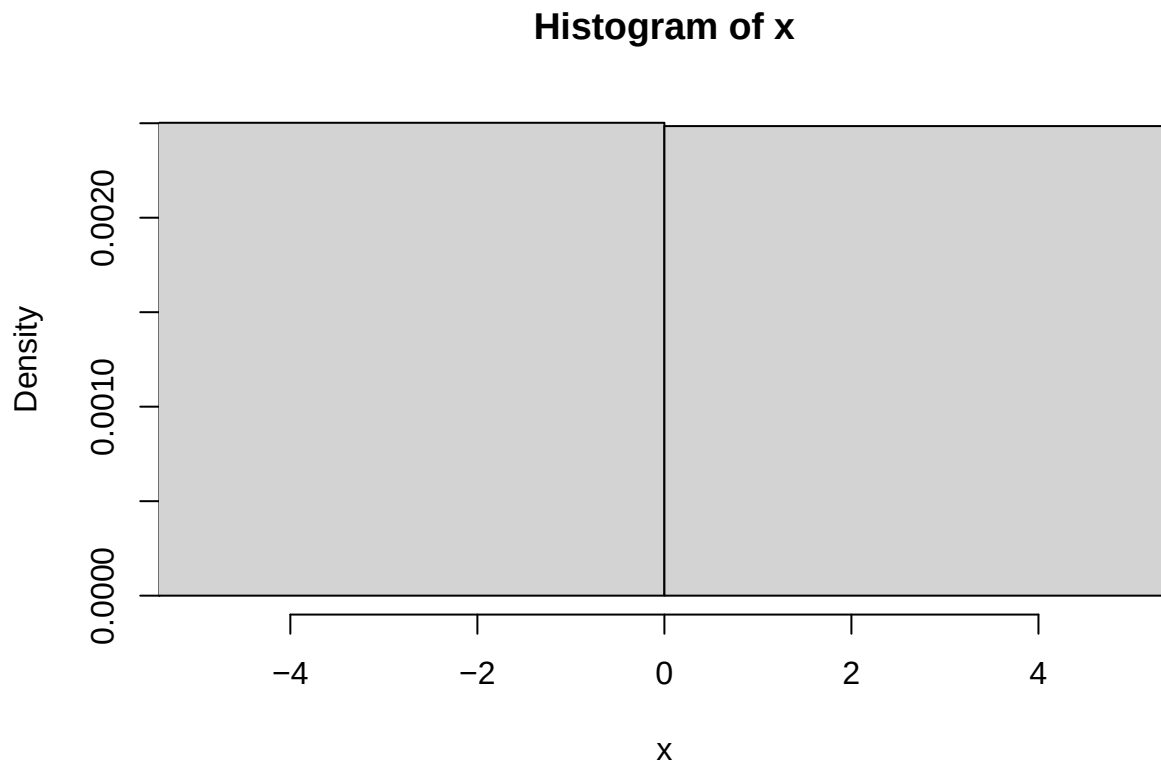
```
## The variance of the generated numbers is 0.201093
```

```
# The above summary statistics are very close to the ones calculated
# analytically, and would most likely grow closer to the true values with a
# greater sample size n.
```

Problem 2

```
# 2a see attached scan
# 2b
set.seed(3)
n = 10000
u = runif(n) # generate random numbers
```

```
x = tan(pi*(u-1/2)) # apply inverse transformation
hist(x,xlim=c(-5,5),freq=0) # display histogram of generated numbers
z = seq(-5,5,0.01) # horizontal axis
lines(z, 1/pi*(1/(1+z^2)), col='blue') # superimpose pdf
```



```
cat('The mean of the generated numbers is', mean(x), '\n') # summary statistics
```

```
## The mean of the generated numbers is 0.5311797
```

```
cat('The median of the generated numbers is', median(x), '\n')
```

```
## The median of the generated numbers is -0.003822019
```

```
cat('The variance of the generated numbers is', var(x))
```

```
## The variance of the generated numbers is 1757.011
```

Problem 3

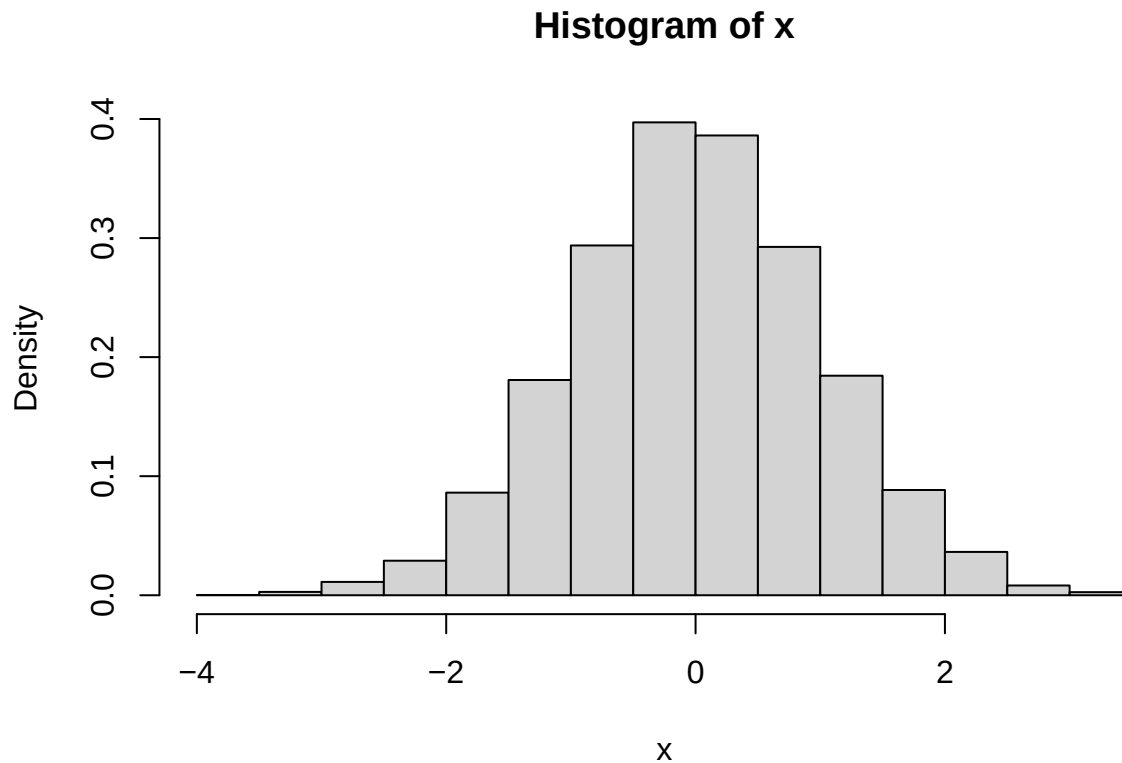
```

# a
# The accept-reject method is a method of generating random numbers when the
# inverse transform method fails to generate the required random variables.
# To implement it, the functional form of density f (the target) must be known
# up to a multiplicative constant M. We then consider a simpler density g (the
# candidate), which has a compatible support to f. The constant M is defined to
# be greater than or equal to f(x) divided by g(x), for all values of x.

# The algorithm is as follows - Step 1: Generate a single y from g and a single
# u from the standard uniform distribution. Step 2: Accept x = y if u is less
# than or equal to f(y)/M*g(y). If the inequality does not hold, generate two
# more random variables (respectively) which do satisfy the condition.

# Speaking more specifically, one would want to specify the quantity of random
# numbers desired, and implement a while loop to keep track of how many have
# been accepted, only stopping once the number of accepted equals the total
# number desired. For the algorithm itself, the variables would be generated
# within a while loop, then a conditional statement would be used to check if
# the inequality holds. M would also have to be determined, which can be done
# by plotting the function f(x)/g(x), and determining its maximum. This could
# be achieved through calculus-based methods as well.
n = 10000
k = 0 # accepted numbers
ii = 0 # number of iterations
x = NULL # empty vector of x values
M = 2.11 # M obtained from graph of f(x)/g(x), maximum value
while(k < n) { # loop until n numbers have been accepted
  u = runif(1) # generate single random number
  uy = runif(1) # r.v. for generating y
  y = tan(pi*(uy-1/2)) # apply inverse transformation for Cauchy pdf
  ii = ii+1 # update iterations
  if (sqrt(pi/2)*exp((-y^2)/2)*(1+y^2) / M >= u) { # accept/reject algorithm
    k = k+1 # increase number of successes
    x[k] = y # create x with only accepted values
  }
}
hist(x,freq=0) # histogram of random numbers, should appear normal

```



```
# b
cat('Number of iterations, or amount of random numbers generated:', ii, '\n')

## Number of iterations, or amount of random numbers generated: 21092

# Since the number of iterations required follows a Geometric distribution with
# parameter 1/M, in other words, iterations I ~ Geo(1/M) the expected value
# would be M iterations for each random number. Thus, the expected value for
# 10000 random numbers would be E(10000*I) = 10000*E(I) = 10000*M. In this
# problem, M was chosen as 2.11, so the expected value of all iterations
# is 2.11*10000, which is slightly less than the actual number of iterations.
# c
cat('Sample mean of the random numbers:', mean(x), '\n') # should be close to 0

## Sample mean of the random numbers: 0.006154784

cat('Variance of the random numbers:', var(x)) # should be close to 1

## Variance of the random numbers: 0.9919344

# both sample measures would be closer to the expected measures with a larger
# sample size (i.e. greater values of n)
```